CBNU

MegaTrain

단일 GPU 로 100B+ 모델 학습

Full Precision Training of 100B+ Parameter Large Language Models on a Single GPU

논문 세미나 | arXiv v1 · 2026.04.06

Zhengqing Yuan 외 3 인 · University of Notre Dame / Lehigh University

발표 흐름

핵심 질문

GPU 한 개로
큰 모델의 학습 상태를
어떻게 다룰까 ?

1. 문제와 배경 · 03–08

- LLM 학습 메모리와 CPU–GPU 계층
- 호스트 중심 설계가 필요한 이유

2. MegaTrain 시스템 · 09–18

- 층별 스트리밍과 블록 재계산
- Double buffering 과 stateless templates

3. 실험과 해석 · 19–35

- 모델 크기 · 깊이 · 폭 · 문맥 길이 확장
- 하드웨어 검증 , 수치 주의점 , 토론

논문 한눈에 보기 : 호스트 중심 학습

- 목표 : 제한된 GPU 메모리에서 대규모 모델의 학습 수행
- 설계 : 파라미터 · 기울기 ·Adam 상태를 CPU 메모리에 저장
- 실행 : 층별 스트리밍 + double buffering + stateless templates
- 보고 결과 : H200 한 개와 1.5 TB 호스트 메모리에서 최대 120B
- 추가 결과 : GH200 의 14B 에서 ZeRO-3 대비 1.84×, 7B 의 512K 문맥 학습

Yuan et al. · arXiv:2604.05091v1 · pp.1–2, Abstract, §1

학습 메모리 : 파라미터보다 상태가 더 큼

지속 상태 ≈ (2 + 2 + 4 + 4) × P = 12P bytes

- BF16 가중치 2 B + BF16 기울기 2 B / 파라미터
- FP32 Adam 모멘트 m, v: 각각 4 B / 파라미터
- 70B × 12 B = 최소 840 GB 의 지속 상태
- 여기에 활성값 · 체크포인트 · 연산 workspace 메모리 추가

Yuan et al. · arXiv:2604.05091v1 · p.2, §2.1

정밀도와 모델 규모를 구분해서 읽기

- 논문 제목의 표기 : Full Precision Training
- 본문의 상태 구성 : BF16 가중치 · 기울기 , FP32 Adam 모멘트
- 정밀도 해석 : 실제 BF16/FP32 구성을 함께 명시
- 120B 실험 모델 : GPT-OSS-120B, MoE(전문가 혼합) 의 총 파라미터 수
- 세미나 해석 : dense 모델과 MoE 모델의 계산량 · 상태 구성을 구분

Yuan et al. · arXiv:2604.05091v1 · pp.2, 5, 9, §2.1, Table 2, Appendix A.1

메모리 계층 : 용량과 전송 속도의 교환

계층 / 연결	H200 계열 (Table 1)	GH200 (Table 1)
GPU HBM	141 GB · 4.8 TB/s	96 GB · 4.0 TB/s
호스트 메모리	DDR5 · 2–4 TB	LPDDR5X · 480 GB
호스트 메모리 대역폭	약 200 GB/s	512 GB/s
CPU–GPU 링크	PCIe Gen5 · 128 GB/s	NVLink-C2C · 900 GB/s
NVMe SSD	10+ TB · 5–14 GB/s	10+ TB · 5–14 GB/s

- 일반 계층 사양과 실제 평가 장비를 구분

  - Table 1 은 일반 계층 사양 ; H200 평가 장비는 PCIe Gen4 (§4.1)
  - 링크 peak, 호스트 메모리 대역폭 , 실효 전송 속도는 서로 다른 값

Yuan et al. · arXiv:2604.05091v1 · pp.2, 4, Table 1, §2.2, §4.1

설계 전환 : CPU 가 주 저장소가 됨

기존 GPU 중심 관점

GPU: 모델 상태의 중심

- 모델 상태의 GPU 상주를 기준으로 실행
- CPU 메모리는 offload 공간으로 사용
- GPU 메모리 한계가 확장의 병목

MegaTrain 의 호스트 중심 관점

CPU: 지속 상태의 주 저장소

- CPU 에 파라미터 · 기울기 ·Adam 저장
- GPU 는 현재 층을 실행하는 캐시
- 필요한 가중치를 공급하고 회수

- 상태의 위치와 GPU 의 역할을 함께 바꿈

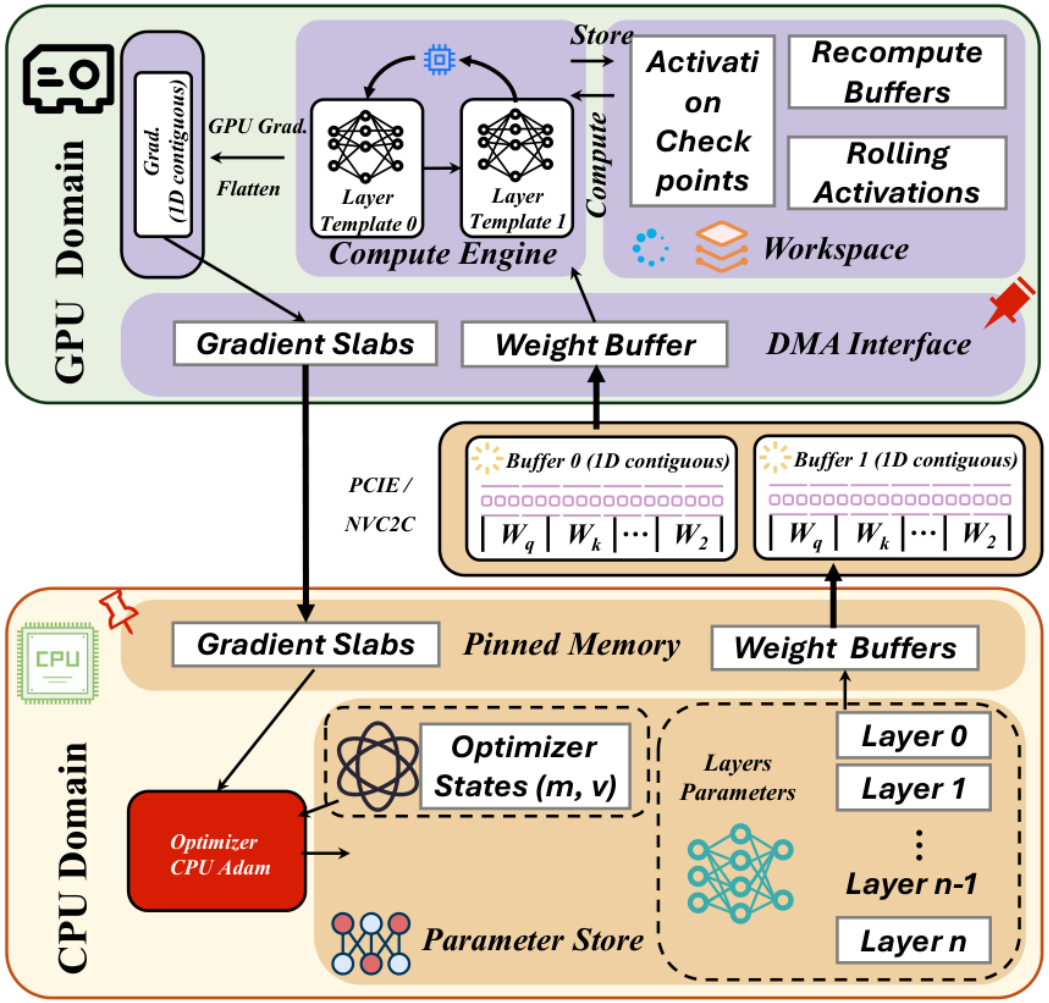
Yuan et al. · arXiv:2604.05091v1 · pp.1–3, §1–3.1

실행 과제 : 저장 이후에 해결할 세 가지

- 전송 병목 : 층마다 가중치를 CPU 에서 GPU 로 가져와야 함
- 실행 의존성 : 가중치 준비 · 역전파 완료 · 버퍼 재사용 순서를 보장
- 텐서 수명 : 필요가 끝난 가중치 · 기울기 · 활성값을 즉시 해제
- 재계산 비용 : 활성값 저장을 줄이는 대신 추가 forward 수행
- 목표 : 작은 작업 집합을 유지하면서 전송을 계산과 중첩

Yuan et al. · arXiv:2604.05091v1 · pp.2–4, §2.3, §3

시스템 아키텍처 : 두 메모리의 역할 분리



- **CPU store ↔ GPU execution cache**

- CPU: 파라미터 , 누적 기울기 , Adam 모멘트를 보관
- GPU: weight buffers, layer templates, activation workspace
- Pinned staging: 가중치 H2D(CPU→GPU), 기울기 D2H(GPU→CPU) 전송
- CPU Adam 이 호스트의 master 상태를 갱신

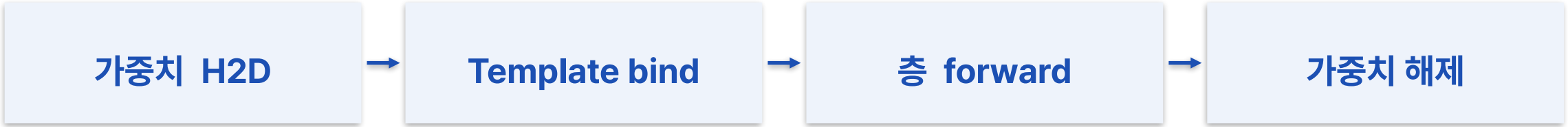
Yuan et al. · arXiv:2604.05091v1 · p.3, Figure 2, §3.1

한 학습 스텝 : 네 단계로 실행

1. Forward	층별 가중치 스트리밍과 계산 K 층 간격으로 활성값 checkpoint 저장
2. Loss	최종 loss 와 출력 기울기 계산 LM head 기울기를 호스트로 반환
3. Backward	블록 재계산 → 층별 local backward 각 층의 기울기를 CPU 로 offload
4. CPU Adam	호스트의 파라미터 · 모멘트 갱신 일부 호스트 작업을 GPU 실행과 중첩

Yuan et al. · arXiv:2604.05091v1 · p.3, Algorithm 1, §3.1

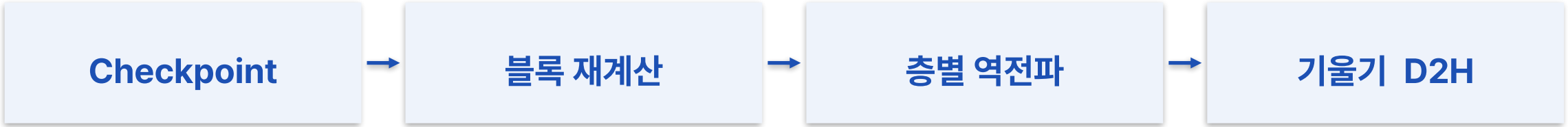
Streaming forward: 계산 후 가중치 해제



- 다음 층의 가중치를 미리 받아 계산 대기 시간을 줄임
- K 층 간격의 활성값 checkpoint 는 GPU 에 유지
- 현재 층 사용이 끝나면 weight buffer 를 다음 층에 재사용
- 전체 모델의 가중치를 GPU 에 동시에 올릴 필요를 줄임

Yuan et al. · arXiv:2604.05091v1 · p.3, §3.1, Algorithm 1

Block-wise backward: 재계산하고 반환



- 마지막 블록부터 시작하여 checkpoint 의 활성값을 복원
- 블록 내부를 forward 순서로 다시 계산해 필요한 활성값 확보
- 층을 역순으로 실행하고 가중치와 활성값의 수명을 종료
- 기울기는 host slab 으로 보내고 CPU master store 에 누적

Yuan et al. · arXiv:2604.05091v1 · p.3, §3.1, Algorithm 1

체크포인트 간격 K: 저장과 재계산의 균형

체크포인트 저장량의 주요 항 : N × Amax × (L / K)

- N: 배치의 토큰 수 · Amax: 층 활성값 크기 · L: 층 수
- K 를 키우면 저장해야 하는 checkpoint 의 개수 감소
- 동시에 한 번에 재계산하는 블록의 길이는 증가
- 실제 GPU 메모리는 활성값 · 연산 버퍼와 함께 평가

Yuan et al. · arXiv:2604.05091v1 · pp.2–3, §2.1, §3.1

Double buffering: 전송과 계산을 중첩

역전파의 중첩 예시 · 실제 시간 비율을 나타내는 도식은 아님



핵심 조건 : 다음 층 가중치 전송 시간 ≤ 현재 층 계산 시간

Yuan et al. · arXiv:2604.05091v1 · pp.4–5, Figure 3, §3.2; 개념도 재구성

CUDA events: 버퍼 사용 순서를 보장

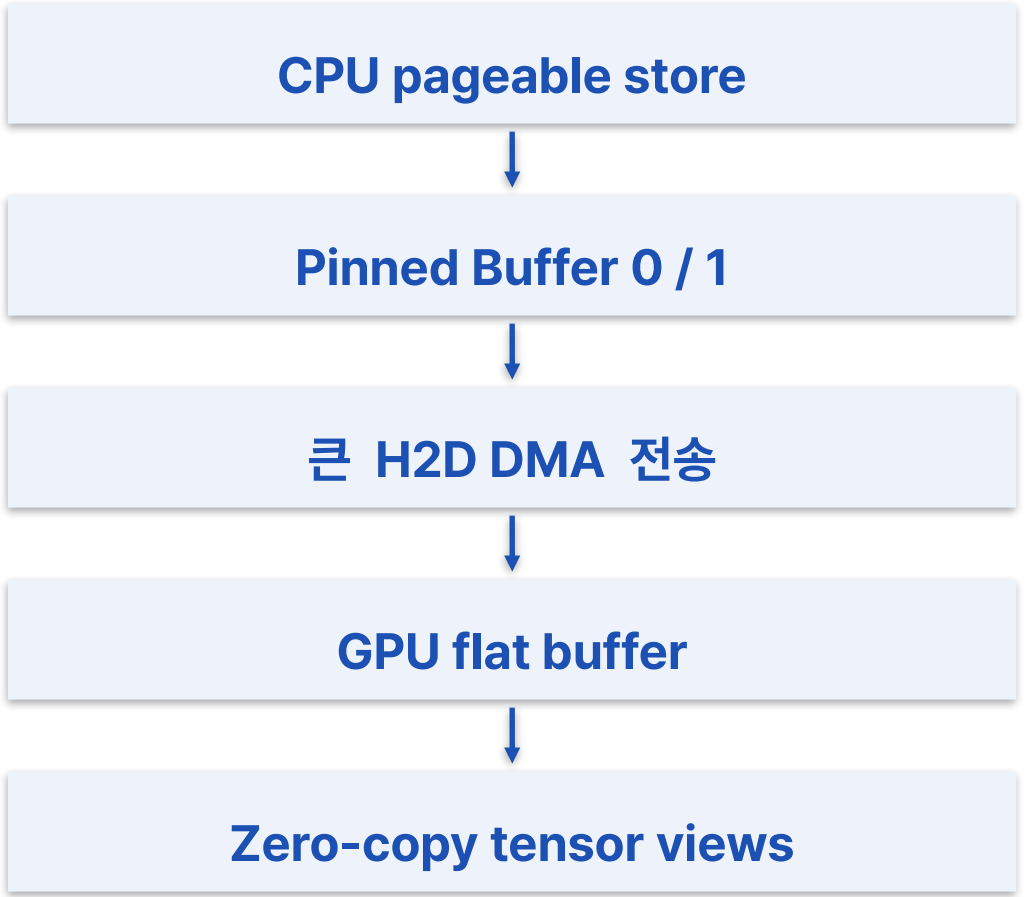
이벤트	기록 시점	다음 동작
Weights-Ready	H2D 전송 완료	Compute 가 Bind / 계산 시작
Backward-Done	층 기울기 생성 완료	D2H 가 기울기 전송 시작
Buffer-Free	D2H 전송 완료	해당 버퍼를 다음 전송에 재사용

- 명시적인 의존성으로 비동기 실행을 연결

- Compute · Weight-Transfer · Gradient-Transfer 의 세 CUDA stream 사용
- 버퍼 재사용 전 완료 이벤트를 확인하여 전송 중인 데이터 수명을 보장

Yuan et al. · arXiv:2604.05091v1 · pp.4, 10, §3.2, Appendix A.2

연속 메모리 배치 : 층별 큰 DMA 전송



- **Layer-contiguous tiling**
  - 한 층의 텐서를 flat block 으로 묶고 4 KB 페이지에 정렬
  - CPU pageable store → 작은 pinned staging pool 로 JIT packing
  - GPU 에서는 flat buffer 의 offset 으로 tensor view 생성
  - 전체 모델 pinning 대신 최대 층 크기의 staging buffers 재사용

Yuan et al. · arXiv:2604.05091v1 · pp.4, 9–10, §3.3, Appendix A.1/A.3

Stateless templates: 구조와 데이터를 분리

지속 상태를 전제하는 전역 그래프

- 전역 autograd 는 역전파를 위한 상태와 의존성을 유지
- 층별 가중치 교체와 텐서 해제에는 추가 수명 제어 필요
- 정적인 CUDA graph capture 를 실행 전제로 사용하지 않음

스트리밍 버퍼에 동적 바인딩

Template A / B

Attention·MLP 연산 구조 재사용

Bind

현재 flat buffer 의 view 연결

Compute / Release

층 실행 후 transient 상태 정리

- StreamIn → Bind → Compute → Offload 를 명시적으로 제어

Yuan et al. · arXiv:2604.05091v1 · pp.4, 10–11, §3.4, Appendix A.4

GPU 메모리 경계 : 현재 작업 집합이 핵심

- 가중치 버퍼 : 전체 파라미터 수보다 최대 층의 크기에 의해 결정
- 추가 GPU 상태 : 활성값 , checkpoint anchors, 재계산 workspace
- 토큰 수 · 층 폭 ·checkpoint 간격에 따라 실제 GPU 사용량 변화
- 호스트 : 전체 지속 상태와 staging 공간을 함께 확보해야 함
- 세미나 해석 : 모델 상태의 상주 부담을 호스트로 이동하는 설계

Yuan et al. · arXiv:2604.05091v1 · pp.2–4, 10–11, §2.1, §3, Appendix A.3

실험 장비 : GPU 는 하나 , 호스트 용량은 다름

평가 플랫폼	GPU / 호스트 메모리	CPU–GPU 연결
GH200 superchip 1 개	96 GB / 480 GB	NVLink-C2C 900 GB/s peak (양방향)
H200 SXM 1 개	141 GB / 1.5 TB	PCIe Gen4

- 모델 규모별 결과는 플랫폼 조건과 함께 해석

- 7B–32B: GH200 · 72B–120B: H200 (§4.2)
- Table 1 의 PCIe Gen5 일반 사양과 H200 의 실제 평가 연결을 구분

Yuan et al. · arXiv:2604.05091v1 · pp.4–5, §4.1–4.2

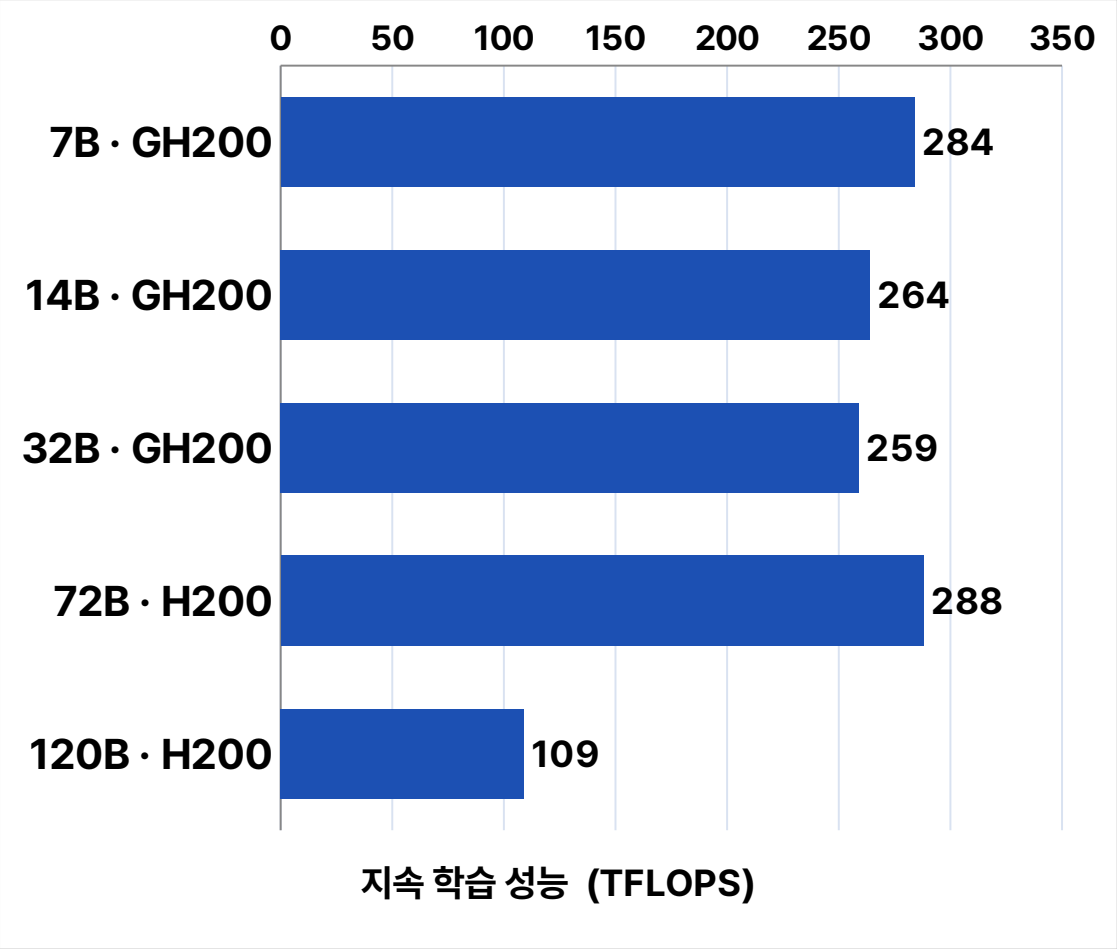
실험 모델과 데이터 : dense 부터 MoE 까지

모델 (Table 2)	총 파라미터	Transformer 층 수
Qwen2.5-7B	7B	28
Qwen2.5-14B	14B	48
Qwen2.5-32B	32B	64
Qwen2.5-72B	72B	80
GPT-OSS-120B (MoE)	120B	36

- 정확도 평가 : MetaMathQA 의 exact match
  - 약 395,000 개 문항을 train 70% / test 30% 로 무작위 분할
  - ZeRO-3 Offload, ZeRO-Infinity, PyTorch Native 등과 조건별 비교

Yuan et al. · arXiv:2604.05091v1 · pp.4–5, §4.1, Table 2

모델 크기별 성능 : 120B 까지 학습 실행

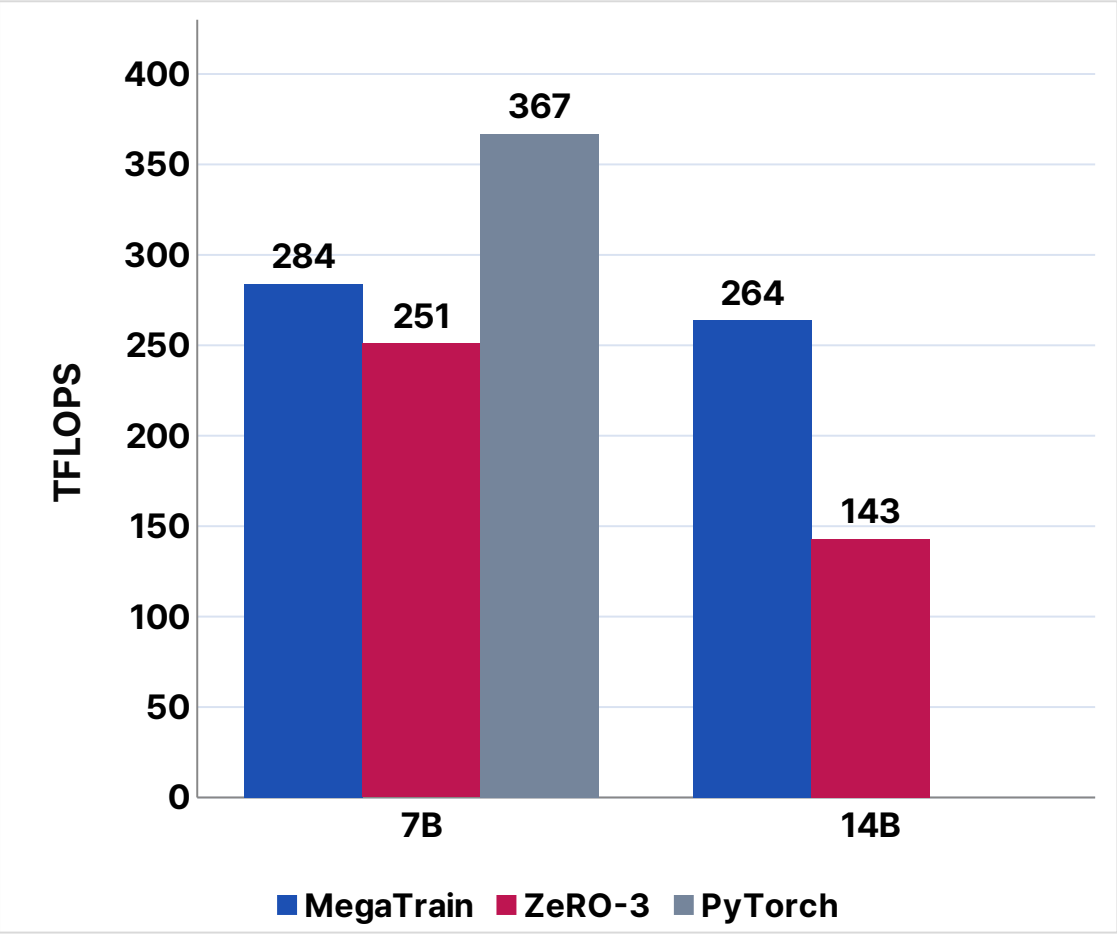


- **Figure 1 의 지속 학습 TFLOPS**

- GH200: 7B 284, 14B 264, 32B 259 TFLOPS
- H200: 72B 288, 120B 109 TFLOPS
- 120B 는 GPT-OSS MoE 의 총 파라미터 규모
- 장비와 모델 구조가 바뀌므로 규모 간 수치를 함께 해석

Yuan et al. · arXiv:2604.05091v1 · pp.1, 5, Figure 1, §4.2

GH200 기준선 비교 : 14B 에서 1.84× 보고



- 빠른 실행과 가능 범위를 함께 평가

• 14B: MegaTrain 264 vs ZeRO-3 143 TFLOPS; 논문은 1.84× 보고

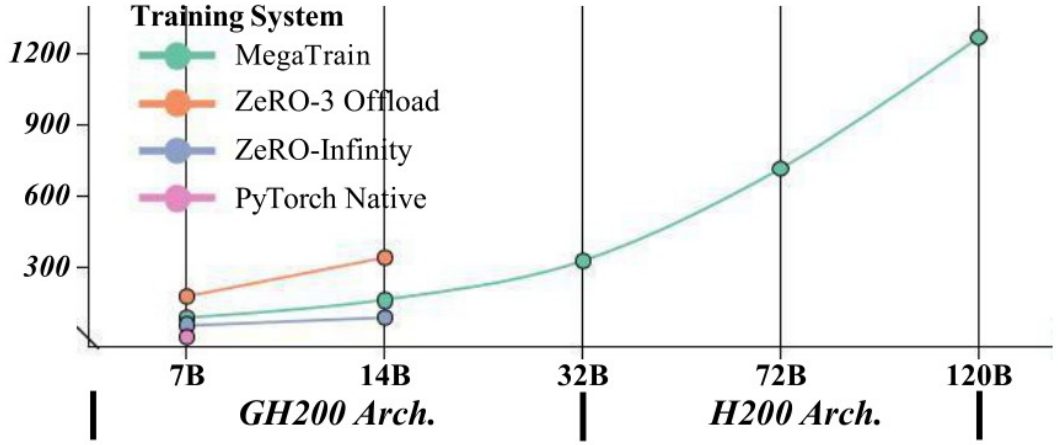
• 7B: PyTorch Native 367 TFLOPS 가 MegaTrain 284 보다 높음

• ZeRO-Infinity: 7B 42, 14B 44 TFLOPS (Figure 1)

• 32B: Figure 1 의 기준선은 OOM, MegaTrain 은 259 TFLOPS

Yuan et al. · arXiv:2604.05091v1 · pp.1, 5, Figure 1, §4.2

호스트 메모리 : 확장 한계가 이동



축 : 모델 크기 / 호스트 메모리 (GB)

• Figure 4 · 원문 그래프

- 파라미터 · 기울기 ·optimizer 상태는 호스트 용량을 사용
- 저자 설명 : 연속 배치와 master store 로 보조 복제와 단편화 감소
- 7B–32B: 480 GB host,
72B–120B: 1.5 TB host 에서 평가
- 세미나 해석 : GPU 개수와 함께 호스트 메모리 비용도 평가

Yuan et al. · arXiv:2604.05091v1 · p.5, Figure 4, §4.2

정확도 : offload 기준선과 유사한 결과

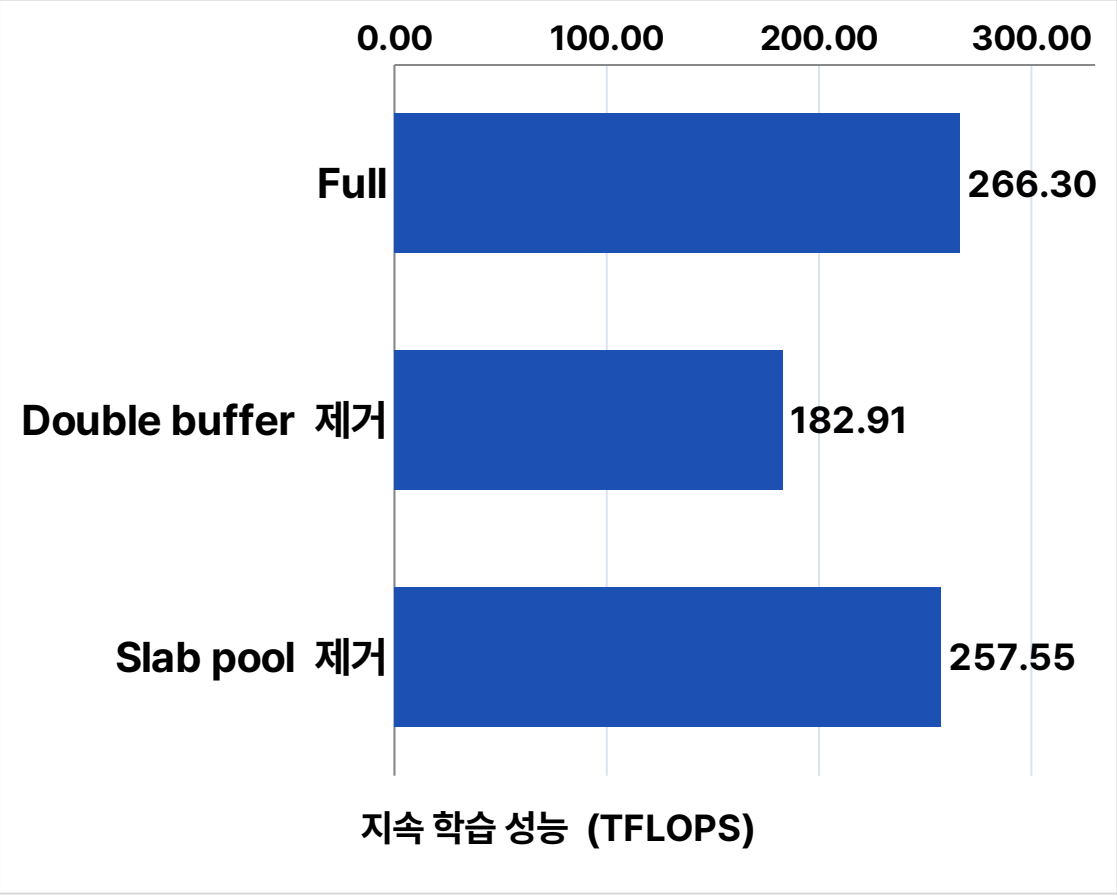
학습 시스템 (Table 3)	7B 정확도 (%)	14B 정확도 (%)
Baseline (원문 표기)	33.47	37.58
ZeRO-3 Offload	88.93	92.41
ZeRO-Infinity	88.97	92.36
PyTorch Native	88.91	–
MegaTrain	88.99	92.52

- 상태 배치 변경 후에도 최종 정확도를 유지

- ZeRO-3 와 차이 : 7B +0.06%p, 14B +0.11%p (표 수치로 계산)
- 최종 exact-match 결과에 대한 근거 ; "–" 는 원문에서 미보고

Yuan et al. · arXiv:2604.05091v1 · p.5, Table 3, §4.2

Ablation: double buffering 이 핵심



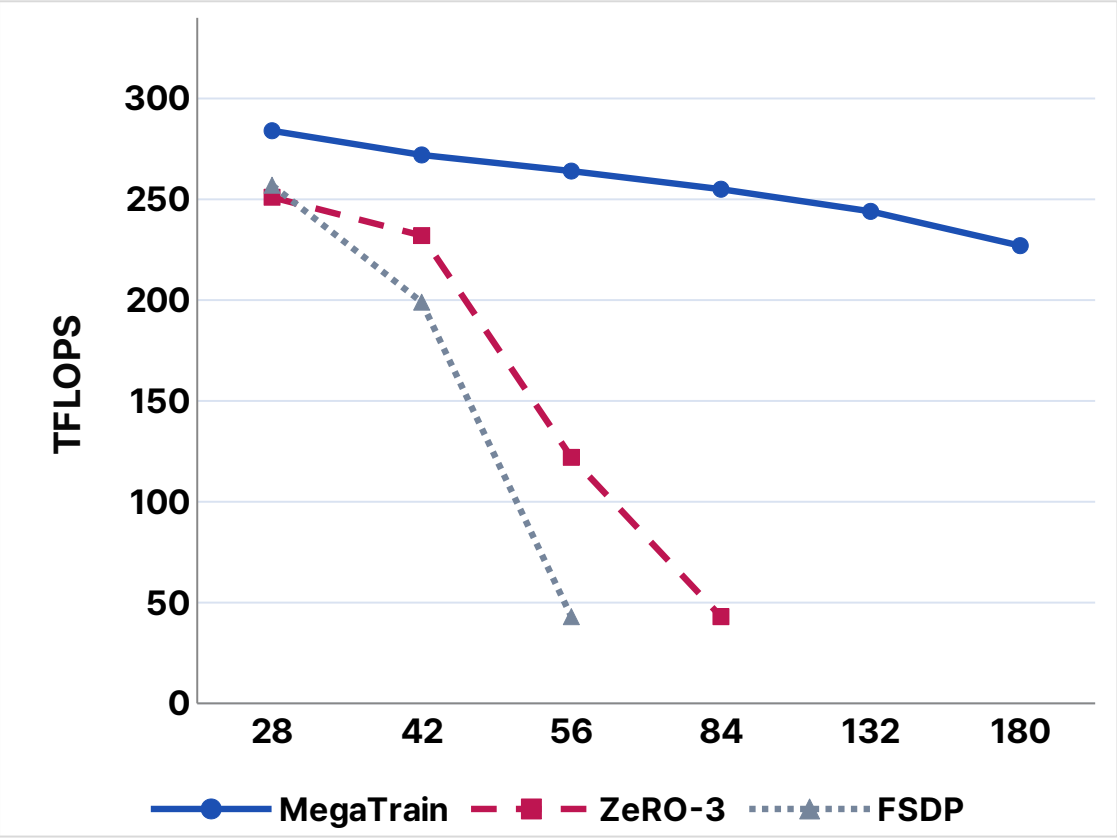
- Table 4 · 같은 BS=96 조건

- Full: 266.30 TFLOPS, GPU 메모리 75.93 GB
- Double buffering 제거 : 182.91 TFLOPS, 31.3% 감소
- Gradient slab pool 제거 : 257.55 TFLOPS, 약 3.3% 감소
- Checkpoint=1 은 표와 본문 수치가 달라 뒤에서 따로 표시

Yuan et al. · arXiv:2604.05091v1 · p.6, Table 4, §4.3; 감소율은 표 수치로 계산

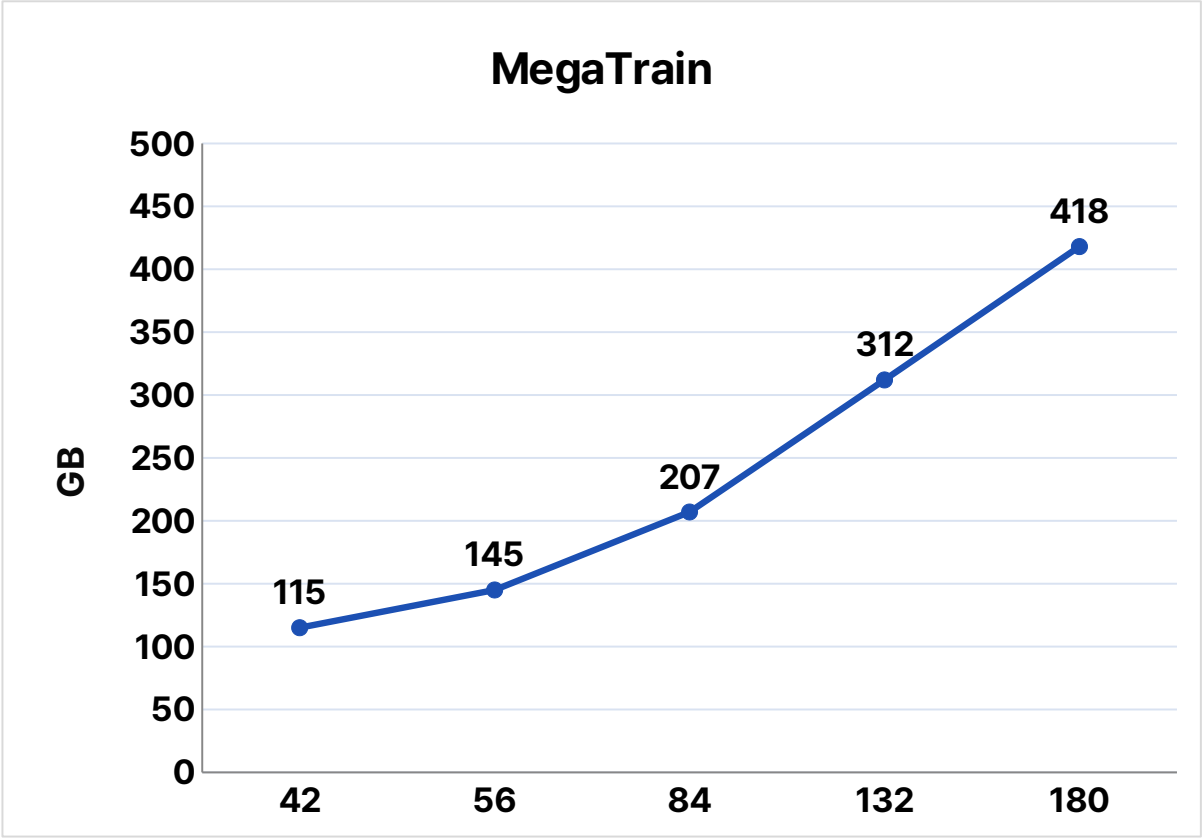
깊이 확장 : 180 층까지 실행

학습 성능 (TFLOPS)



미표시점은 OOM · 층 수에 따라 성능 비교

MegaTrain 호스트 메모리 (GB)



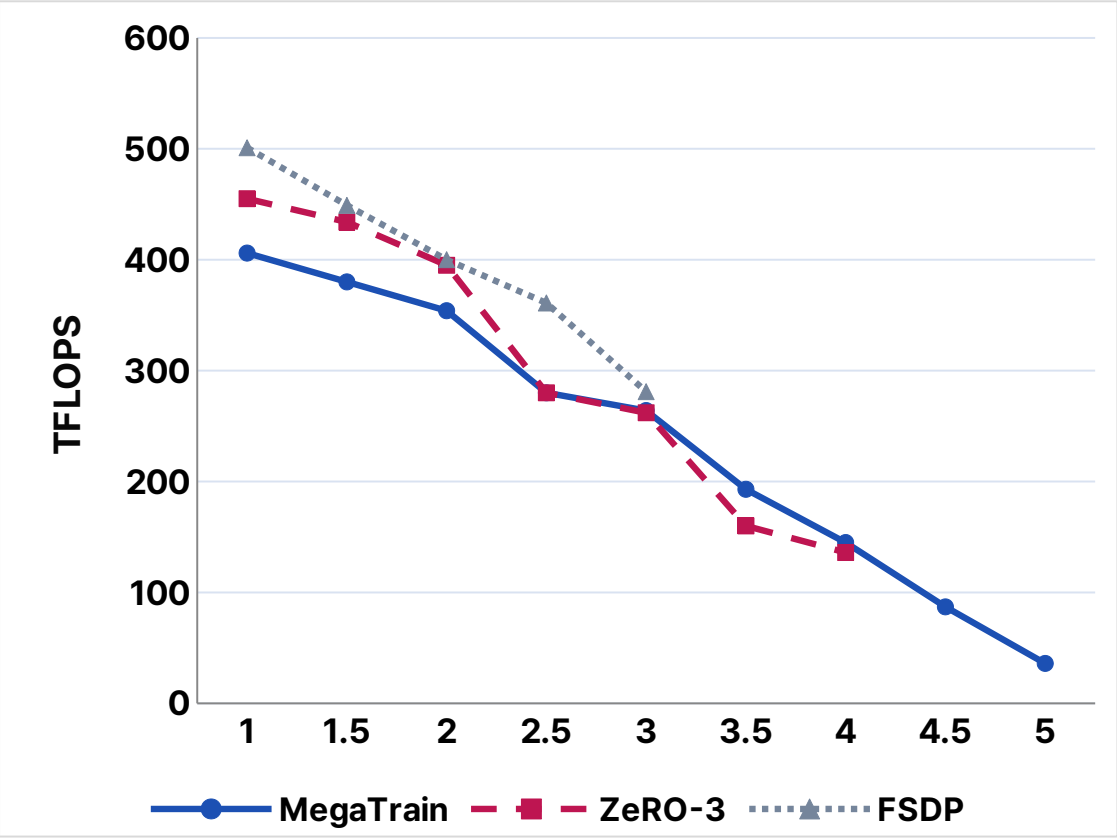
180 층 : 43.04B 파라미터 (Table 5)

- 조건 : GH200 · 층 폭 고정 · Table 5 의 Device Alloc=3.83 GB

Yuan et al. · arXiv:2604.05091v1 · pp.6–7, Table 5, Figure 5, §4.4

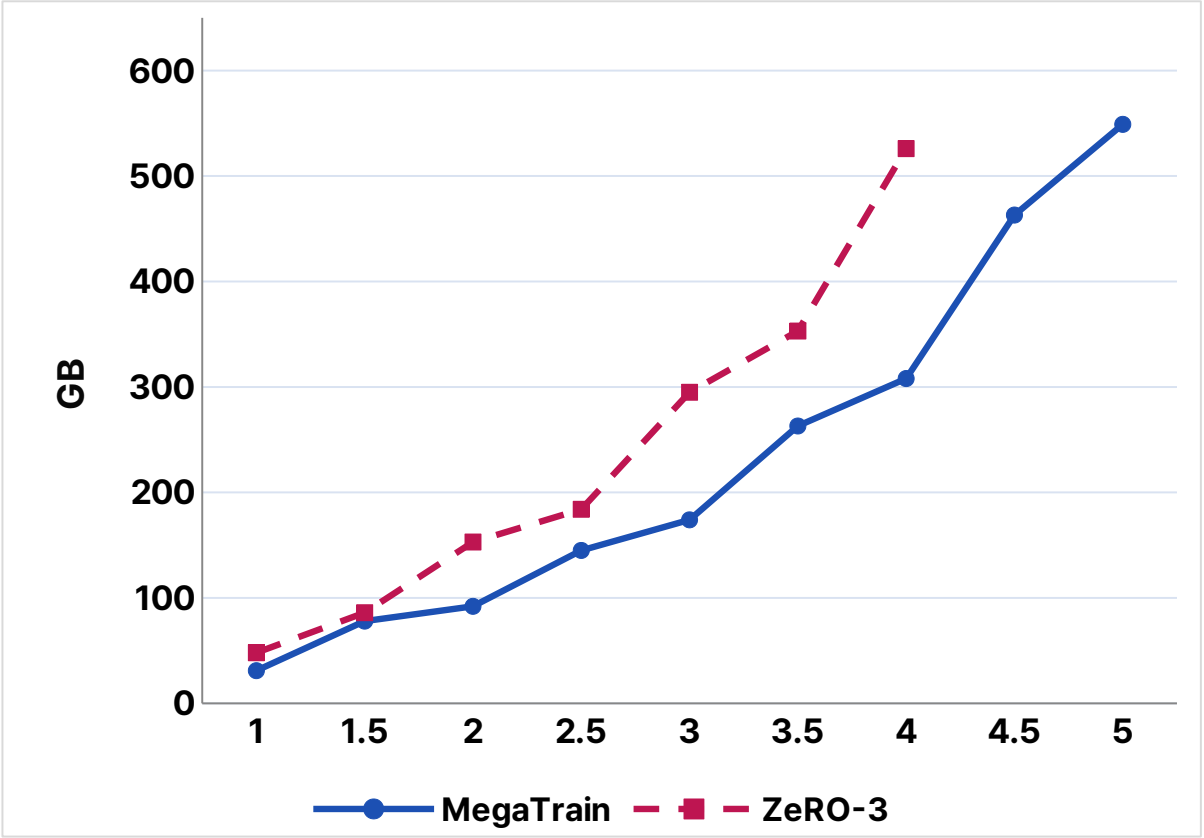
폭 확장 : 큰 층을 처리하는 비용

학습 성능 (TFLOPS)



가로축 : 폭 배율 (×) · 미표시점은 OOM

호스트 메모리 (GB)



층 수 고정 · 작은 폭에서는 기준선이 더 빠름

- 4.5× 부터 기준선 OOM · 5× 에서 MegaTrain 36 TFLOPS / 549 GB host

Yuan et al. · arXiv:2604.05091v1 · pp.6–7, Table 6, Figure 6, §4.5

CBNU

Long context: GH200 에서 최대 512K

문맥 길이	BS	Step (s)	TFLOPS	GPU 메모리 (GB)
1K	158	27.05	284.7	74.2
8K	25	32.36	294.5	86.5
32K	6	32.18	316.7	84.0
128K	1	26.13	305.3	62.1
256K	1	236.1	401.2	88.2
512K*	1	871.4	407.4	81.9

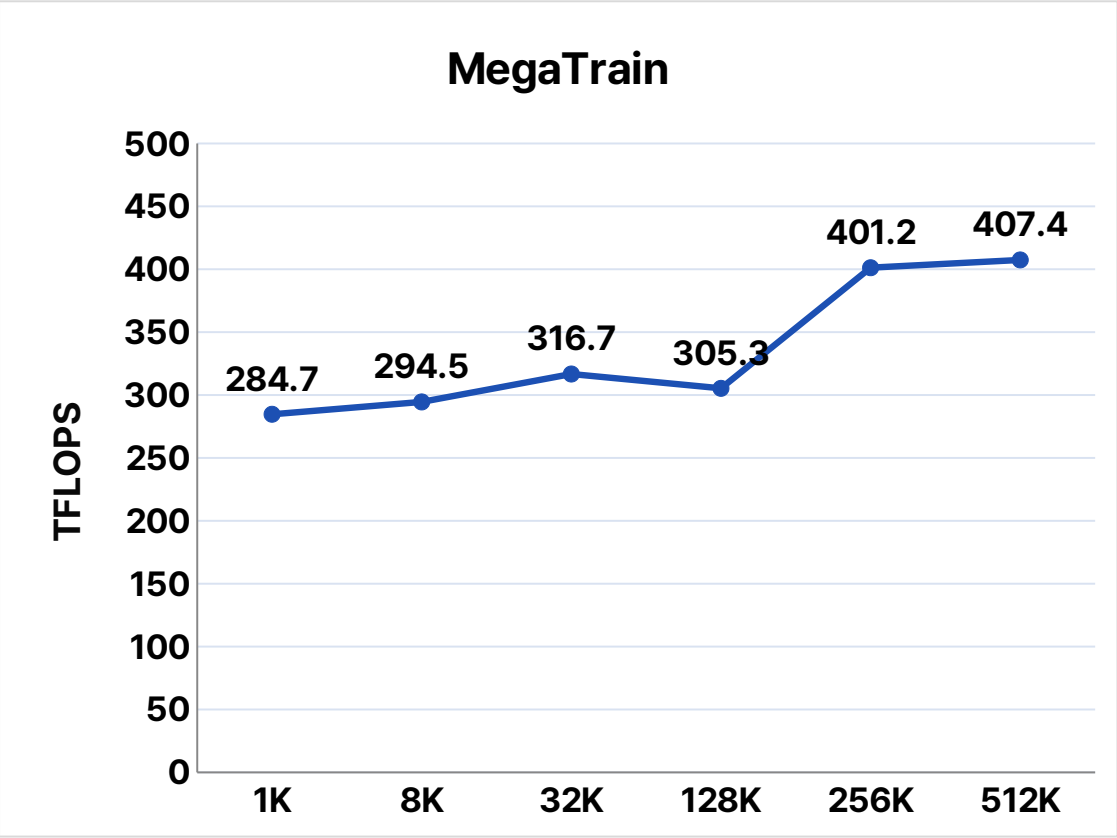
- 512K* 는 chunked MLP execution 을 적용한 결과

- 7B 모델 · GH200 단일 GPU; 각 문맥 길이에서 배치 크기도 달라짐
- 문맥이 길어지면 attention 계산량과 활성값 부담이 함께 증가

Yuan et al. · arXiv:2604.05091v1 · pp.1, 7, Abstract, Table 7, §4.6

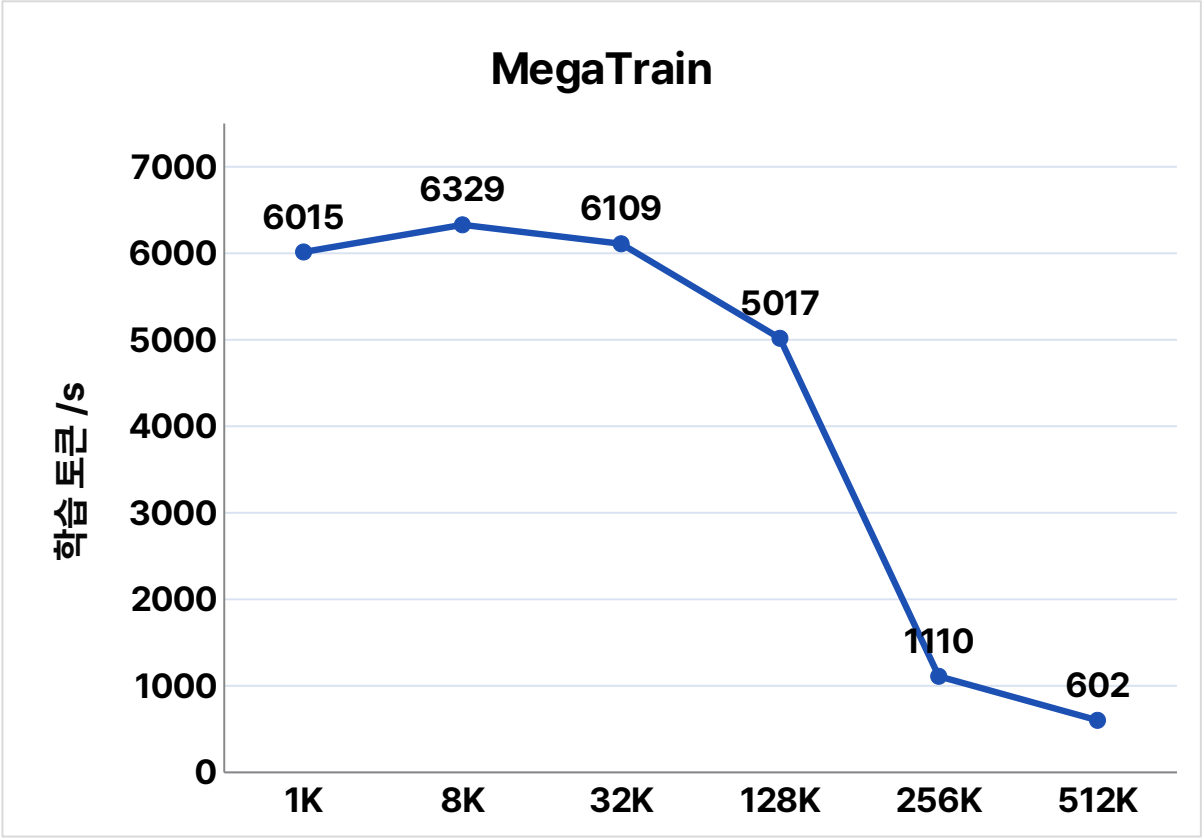
Long context: 성능 지표를 함께 읽기

지속 계산 성능 (TFLOPS)



FLOPs 산식에 attention 의 문맥 길이 ² 항 포함

학습 처리량 (토큰 /s, 근사)

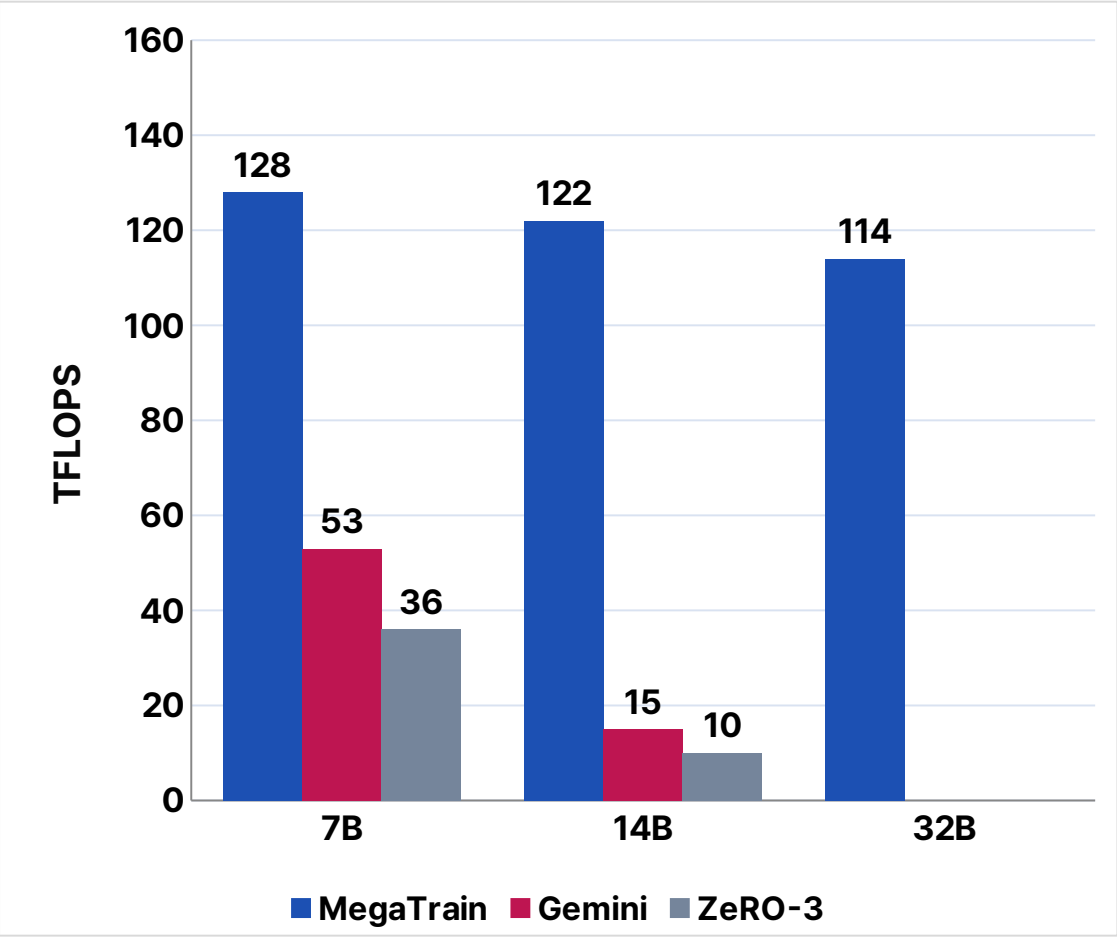


표의 배치 토큰 수로 계산한 학습 처리량

- 512K: 407.4 TFLOPS · 약 602 학습 토큰 /s · 871.4 초 /step

Yuan et al. · arXiv:2604.05091v1 · p.7, Table 7; 토큰 /s = 표의 Tokens / Step(s) 로 계산

A100 PCIe 검증 : 다른 연결에서도 실행



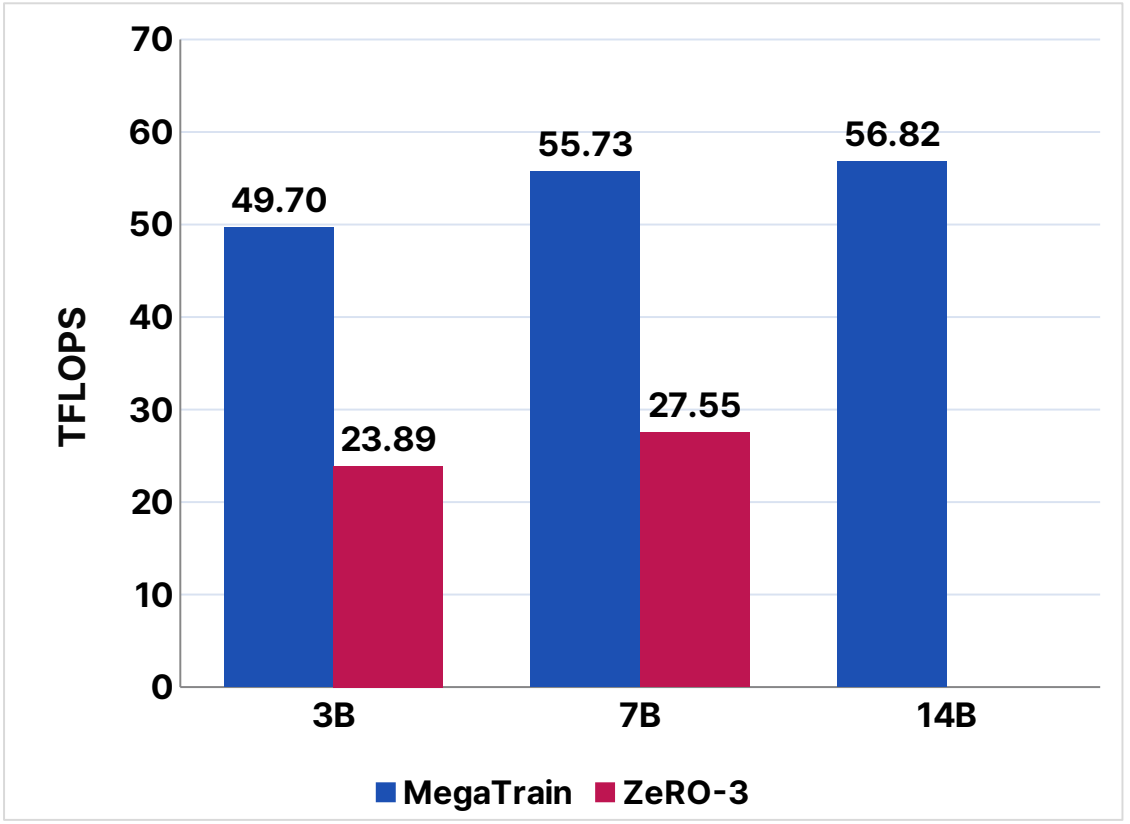
- Figure 7 · 단일 A100 80 GB

- 환경 : host 600 GB DDR4, PCIe Gen4
- 14B: MegaTrain 122, Gemini 15, ZeRO-3 10 TFLOPS
- 14B 의 보고 배율 : Gemini 대비 8.13×, ZeRO-3 대비 12.20×
- 32B: 기준선 OOM, MegaTrain 114 TFLOPS

Yuan et al. · arXiv:2604.05091v1 · p.8, Table 8, Figure 7, §4.7

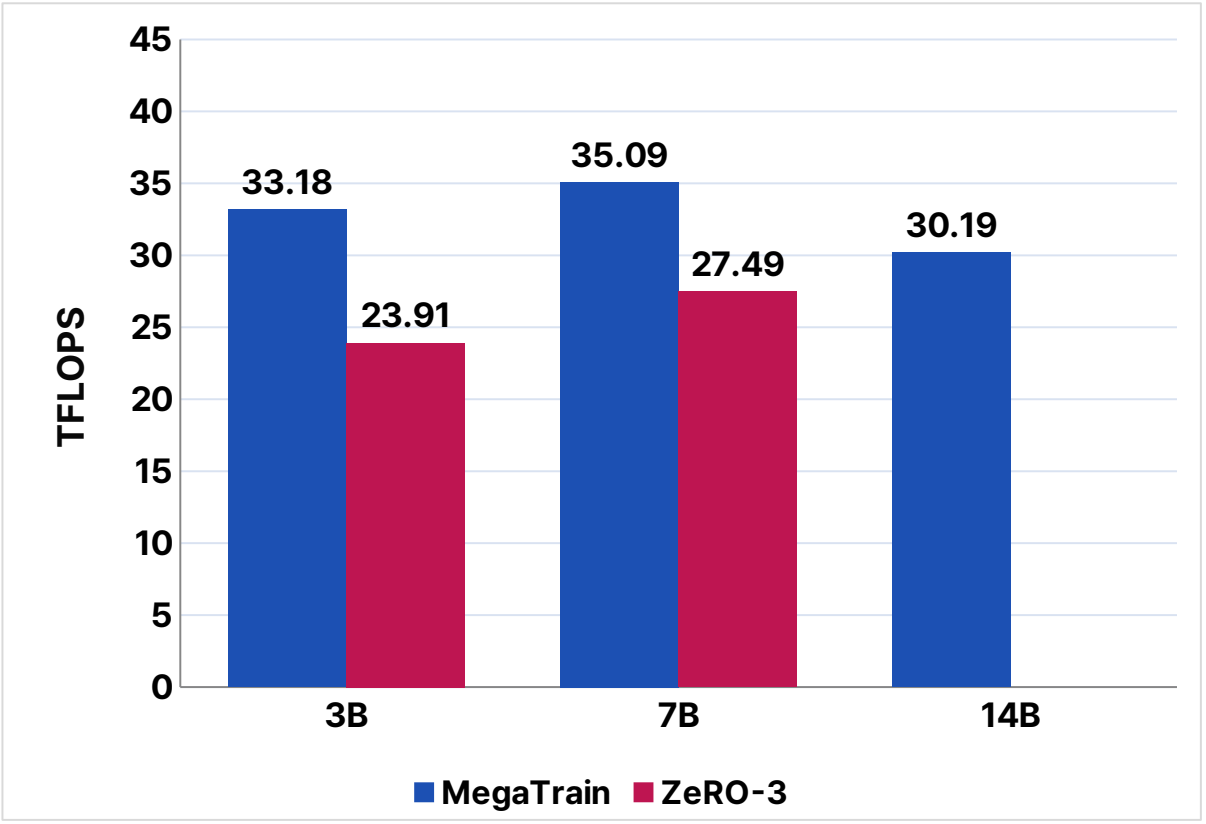
워크스테이션 GPU: 24 GB 에서도 14B 실행

RTX A6000 · 48 GB VRAM



MegaTrain BS: 15 / 12 / 9 · ZeRO-3 14B OOM

RTX 3090 · 24 GB VRAM



MegaTrain BS: 7 / 5 / 3 · ZeRO-3 14B OOM

- 길이 8,192 · ZeRO-3 BS=1 · MegaTrain 은 모델별 최대 BS

Yuan et al. · arXiv:2604.05091v1 · p.8, Tables 8–9, §4.7; 호스트 251 GB, GPU 별 독립 평가

원문 수치 차이 : 표 · 그림과 본문을 구분

- Checkpoint=1: Table 4 는 BS 64 · 240.45 TFLOPS,

§4.3 본문은 BS 32 · 184.2 TFLOPS 로 기술

- 1K long context: Table 7 은 284.7, §4.6 본문은 264.8 TFLOPS

- 깊이 실험 : Table 5 의 28 층은 7.62B,

§4.4 본문은 시작 규모를 10.9B 로 서술

- 본 자료는 표 · 그림의 수치를 우선 인용하고 불일치는 따로 표시

Yuan et al. · arXiv:2604.05091v1 · pp.6–7, Tables 4–5/7, §4.3–4.6; v1 대조

설계의 의미와 결과 해석의 조건

설계의 의미

- 지속 상태의 주 저장소를 CPU 로 전환
- 전송 · 계산 · 반환을 병렬로 실행
- 텐서 수명과 buffer ownership 을 명시적으로 관리
- 큰 모델과 긴 문맥에 실행 범위 확장

실험 해석의 조건

- 대용량 host RAM 과 CPU–GPU 링크가 필요
- 120B 는 MoE 총 파라미터 수
- 최대 BS 비교와 같은 BS 비교를 구분
- Device Alloc 수치를 전체 GPU 사용량과 구분

- 단일 GPU 학습의 가능성을 호스트 비용 · 모델 구조 · 배치 조건과 함께 평가

Yuan et al. · arXiv:2604.05091v1 · §3–4, Appendix A; 오른쪽 항목은 세미나 해석

결론 : 메모리 배치와 실행 스케줄의 결합

- CPU: 파라미터 · 기울기 ·Adam 상태의 주 저장소
- GPU: 현재 층과 재계산 작업을 처리하는 transient cache
- 성능 핵심 : double buffering 으로 전송과 계산을 중첩
- 실행 핵심 : stateless templates 와 이벤트 기반 수명 제어
- 적용 판단 : GPU 용량 , host RAM, 링크 , 모델 · 배치 조건을 함께 고려

Yuan et al. · arXiv:2604.05091v1 · pp.1–8, §3–5; 마지막 항목은 세미나 해석

토론 : 어떤 조건에서 이 설계가 유리할까 ?

- 느린 PCIe 환경에서 계산 · 전송 중첩 조건을 어떻게 맞출까 ?
- 같은 배치 · 문맥 길이에서 어떤 성능 지표로 기준선과 비교할까 ?
- MoE 에서 dense 모델로 바뀌면 층 폭과 host 비용은 어떻게 달라질까 ?
- 원문: Zhengqing Yuan 외 3인, arXiv:2604.05091v1 (2026.04.06)

코드: github.com/DLYuanGod/MegaTrain